Design YouTube

功能性需求（Functional Requirements）

1. 使用者可以上傳影片（upload videos）。
2. 使用者可以觀看（串流，stream）影片。

不在範圍內（Out of Scope）

- 使用者搜尋影片（search for videos）。
- 使用者對影片留言（comment on videos）。
- 使用者查看推薦影片（recommended videos）。

非功能性需求（Non-Functional Requirements）

1. 系統必須具備高可用性（high availability），並以可用性優先於一致性（availability over consistency）。
2. 系統需支援大型影片的上傳與串流（uploading and streaming large videos），單支影片大小可達數十 GB（10s of GBs）。
3. 即使在低頻寬（low bandwidth）環境下，系統仍需提供低延遲（low latency）的影片串流體驗。
4. 系統必須能夠水平擴展（scale），以支援高數量的影片上傳與觀看：
    - 每日約 100 萬（~1M）支影片上傳
    - 每日約 1 億（~100M）次影片觀看

API 設計（API Design）

1. 上傳影片（Upload video）

buildmoat.org

```
POST /videos/upload
Response:
{
  Video,
  VideoMetadata
}
```

- `Video`（影片內容）：上傳的影片資料（實務上通常為 multipart / chunked upload）。
- `VideoMetadata`（影片中繼資料）：例如影片長度、解析度、編碼格式、檔案大小、上傳者資訊等。

2. 串流影片（Stream video）

```
GET /videos/:videoId -> Video & VideoMetadata
```

- `videoId`（影片 ID）：欲播放影片的唯一識別碼。
- 回傳內容包含影片資料與對應的 VideoMetadata（實務上多為回傳播放清單或分段 URL，而非整支影片）。

buildmoat.org

High-Level Design

影片串流相關名詞（Terms about video streaming）

- **Video Codec（影片編解碼器）**：影片的壓縮 / 解壓縮演算法（例如 H.264、MPEG-4）。
- **Video Container（影片容器）**：用來存放影片資料與中繼資料的檔案格式（例如 mp4、avi）。
- **Bitrate（位元率）**：單位時間內傳輸的位元數，通常以 kbps 或 mbps 表示。
- **Manifest files（清單檔）**：文字型文件，用來告訴影片播放器如何取得並播放影片內容。

  串流平台不會傳送單一巨大影片檔，而是將影片切成多個小段（segments）。Manifest 會列出這些片段的 URL，作為播放器串流時的「索引」。

在本設計中，我們使用 **video format** 來泛指 **video codec + container**。

1. 使用者可以上傳影片（Users can upload videos）

為了支援影片上傳，需要考慮以下幾個面向。

要儲存什麼（What to store：video + metadata）

影片資料需符合以下需求：

1. 以多種 formats 儲存，確保能在不同裝置上播放。
2. 切分為多個 segments（每段數秒、可獨立播放），讓使用者在觀看時以串流方式載入，而非一次下載整支影片。

除了影片資料外，還需儲存 metadata（例如 title、description、uploader 等），以便在載入影片時同時呈現相關資訊。

儲存在哪裡（Where to store：DB + Blobstore）

- **VideoMetadata**：

  假設上傳速率約為 ~1M videos/day，則一年約會累積 ~365M 筆紀錄。

  因此應選擇可水平分割（horizontally partitioned）的資料庫，例如 Cassandra。

- **Video data**：

  將大型二進位資料直接存入 DB 是不合適的。

  DB 擅長結構化資料與查詢，但不適合儲存大型 BLOB；例如將 100MB 檔案存成 BLOB 會嚴重影響查詢效能、備份與 replication。

  Blob storage（例如 Amazon S3、Google Cloud Storage（GCS）、Azure Blob Storage）正是為此設計。

經驗法則：

若資料大於 10MB，且不需要 SQL 查詢，通常就該放在 blob storage。

如何儲存（How to store）

- **Metadata**：同樣基於 ~1M uploads/day 的假設，使用可水平擴展的 DB（例如 Cassandra）儲存。
- **Video data**：

  正確做法**不是**由 client 將影片上傳到我們的 service，再由 service 轉傳至 S3。

  相反地，client 應直接將影片上傳至 S3。

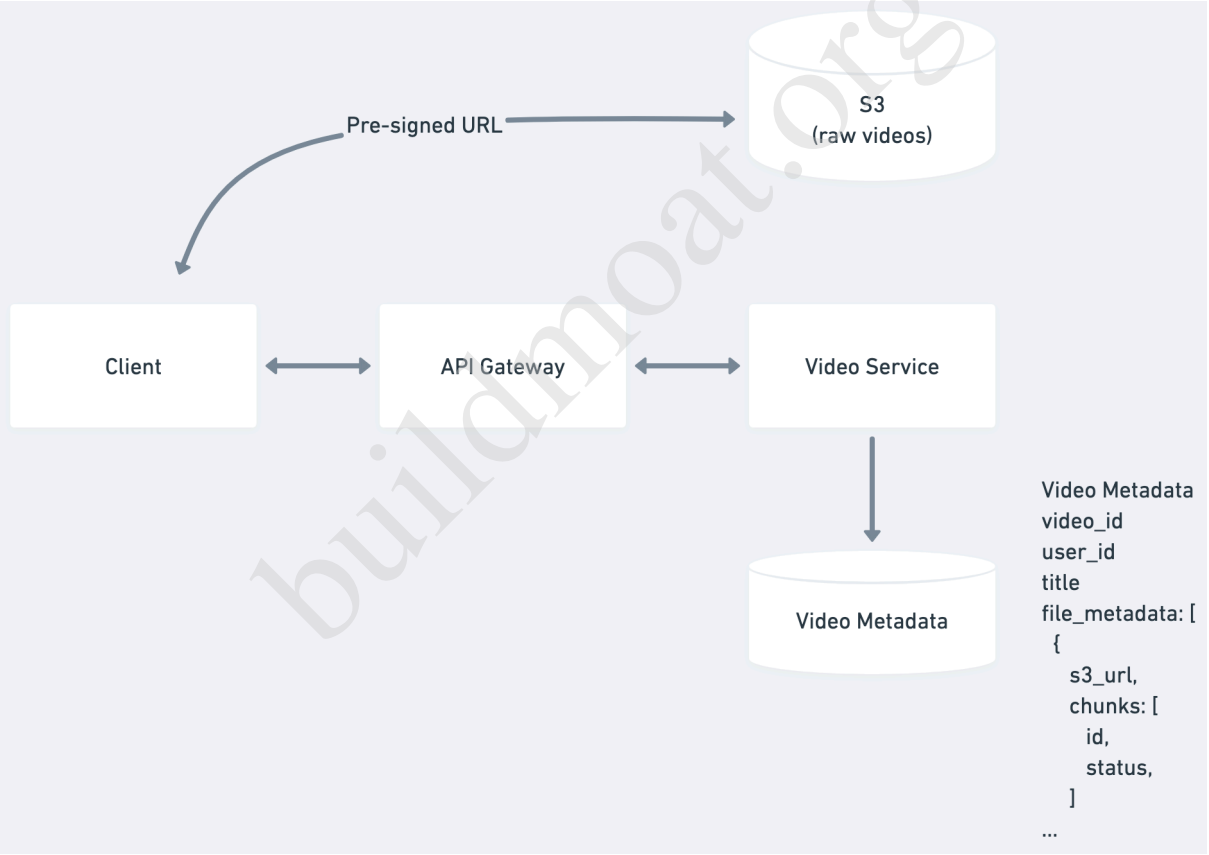
流程如下：

bullmoat.org

1. Client 向我們的 service 發送上傳請求。
2. Service 在 metadata table 建立一筆紀錄，並產生 **presigned URL（預先簽署網址）** 回傳給 client。
   - Presigned URL 是一個**帶有臨時授權的網址**，讓使用者在限定時間內直接對 S3 進行操作（上傳或下載），而不需要雲端帳號或金鑰。

     由 server 使用雲端憑證為特定物件產生。
3. Client 使用 presigned URL，將影片資料直接上傳至 S3。
4. 由於影片檔通常非常大（可達數 GB），上傳時會將檔案切成 chunks，並透過 S3 的 multipart upload 逐段上傳與記錄進度。

   細節將在 deep dive 中說明。



因此，API 也需要調整為向 service 請求 `presigned_url`，而非直接上傳影片：

```
POST /videos/presign_url
Request:
{
```

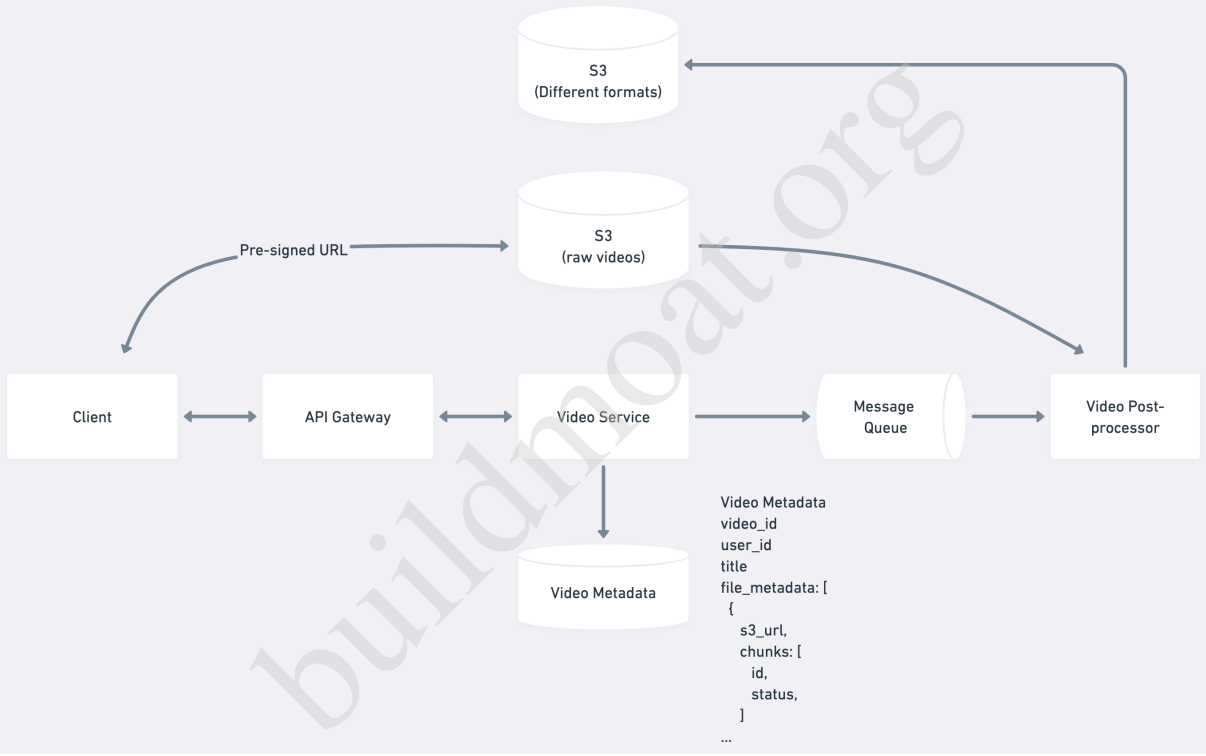
```
  VideoMetadata
} -> presigned_url
```

2. 使用者可以觀看影片（Users can watch videos）

使用者觀看影片時，直接下載整支影片並不可行，那會變成離線下載而非串流。

因此，影片在上傳完成後，會觸發一個 **post-processing** 流程：

- 將影片切成小段 segments
- 轉碼（transcode）成多種 formats，以支援不同裝置與網路條件



Adaptive Bitrate Streaming（自適應位元率串流）

為了提供最佳觀看體驗，常見做法是使用 adaptive bitrate streaming：

1. Client 先取得 VideoMetadata，其中包含指向 S3 上 manifest file 的 URL。
2. Client 下載 manifest file。
3. Client 根據網路狀況與使用者設定，選擇合適的 format，並從 manifest 中取得對應 segment 的 URL，下載第一段。
4. Client 播放該 segment，同時持續下載後續 segments。
5. 若 client 偵測到網路狀況變差（或變好），會即時調整下載的影片品質：

- 網路變差 → 改下載壓縮率更高、解析度更低的 segments
- 以避免播放中斷

這種做法雖然較為複雜，但能提供更穩定的串流體驗，也讓 client 成為系統中更主動的一環。

同時，它也依賴前面的設計決策：**影片分段、儲存多種 formats，以及 manifest files 的產生**。

深入探討（Deep Dives）

1. 後處理（post-processing）如何運作以支援自適應位元率串流（adaptive bitrate streaming）？

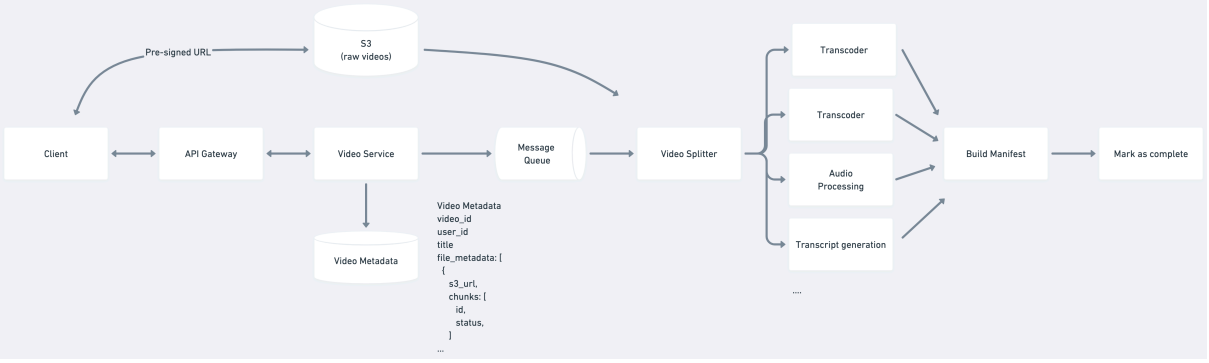
當影片以原始格式上傳後，必須經過後處理，才能以可串流的形式提供給各種不同裝置。如前所述，影片後處理本質上是一個 **pipeline（處理管線）**，其輸出包含：

1. 儲存在 S3 中、以不同 formats（codec + container）編碼的影片 segments。
2. 儲存在 S3 中的 manifest files（一個 primary manifest 與多個 media manifests），其中 media manifests 會引用對應的 segment files。

為了產生 segments 與 manifest files，處理流程可依序分為：

1. 使用工具（例如 **ffmpeg** 或類似工具）將原始影片切分為多個 segments。

   這些 segments 會被轉碼（transcode），並用來產生不同的 video containers。
2. 對每個 segment 進行轉碼，並處理其他相關工作（例如 audio 處理、字幕或 transcript 生成）。
3. 建立 manifest files，引用不同 video formats 下的 segments。
4. 將影片狀態標記為「完成上傳（complete）」。



buildneat.org

上述處理步驟可視為一個 **dependency graph（相依關係圖）**，其中部分步驟（例如對各個 segment 的處理）可平行執行。

由於相依關係是單向的，該圖可被建模為 **DAG（Directed Acyclic Graph，有向無環圖）**。

在 DAG 的編排（orchestration）上，可使用既有技術（例如 Temporal）來負責執行與重試。

2. 如何支援可續傳上傳（resumable uploads）？

在 high-level design 中，我們提到會使用 multipart upload 直接將大型影片上傳至 S3。

這麼做的主要原因之一，是在上傳過程中若發生失敗，可以從中斷處繼續上傳。

使用 multipart upload 的實際流程如下：

1. Server 呼叫 `CreateMultipartUpload`，取得 `UploadId`（**這一步會呼叫 S3**）。
2. Server 在本地為每個 part 產生對應的 `UploadPart` presigned URL（**不需呼叫 S3**）。
3. 對於每個 part：
   a. Client 使用 presigned URL，直接對 S3 發送 `PUT` 請求上傳該 part。
   b. 從回應中讀取 `ETag`。
      - **ETag（Entity Tag，實體標籤）** 是 S3 在物件上傳後產生的字串，用來識別檔案版本。

        常見用途包括：

        1. 檢查檔案是否變更（類似 checksum）。
        2. Client 可透過比對 ETag，確認下載檔案的完整性。
   c. Client 將 `{uploadId, partNumber, etag, size?}` POST 回 Server。
   d. Server：
      i. 呼叫 `ListParts`，比對 eTags 以確認該 part 已成功上傳，並在 metadata 中更新 chunk 狀態。
4. 當所有 parts 上傳完成後：
   a. Client 通知 Server 所有 chunks 已完成上傳。
   b. Server 呼叫 `CompleteMultipartUpload`，並提供所有 `(partNumber, eTag)`，以建立最終的 S3 object，同時更新 metadata 中的最終 S3 位置。

buildmoat.org

依照此流程，Server 會在 metadata 中紀錄每個 chunk 的狀態；

若 Client 中途停止上傳，可重新取得 VideoMetadata，找出已完成的 chunks，並跳過它們以繼續上傳。

3. 如何擴展系統以支援每日大量影片的上傳與觀看？

- **Video Service（影片服務）**

  此服務為 stateless，負責回應 presigned URL 的請求與 video metadata 的查詢。

  可透過 load balancer 水平擴展。

- **Video Metadata（影片中繼資料）**

  可選擇易於水平擴展的 NoSQL。

  使用 `video_id` 作為 partition / sharding key，可將影片平均分散至各 partition。

  若出現 hot videos，可：

  - 增加該 partition 的 replicas
  - 或在 DB 前方加上 cache layer 以處理 hot keys

- **Video Processing Service（影片處理服務）**

  此服務需能支援大量影片處理，並在內部協調 DAG 任務如何分配給 worker nodes。

  實務上通常會有一個內部 queue（雖未在圖中顯示）：

  - 用來吸收上傳高峰（bursts）
  - queue 中的 job 數量可作為 auto-scaling 的依據，以動態增加 worker nodes

- **S3**

  S3 在高流量與大量檔案的情境下具備極佳的擴展性，且支援多區域（multi-region）。

  然而，若使用者距離 S3 所在的 data center 較遠，初始載入時間或串流過程可能會受到影響。

為了解決距離 **S3 資料中心較遠**的使用者可能遇到串流效能問題，我們可引入 **CDN（Content Delivery Network，內容傳遞網路）**。

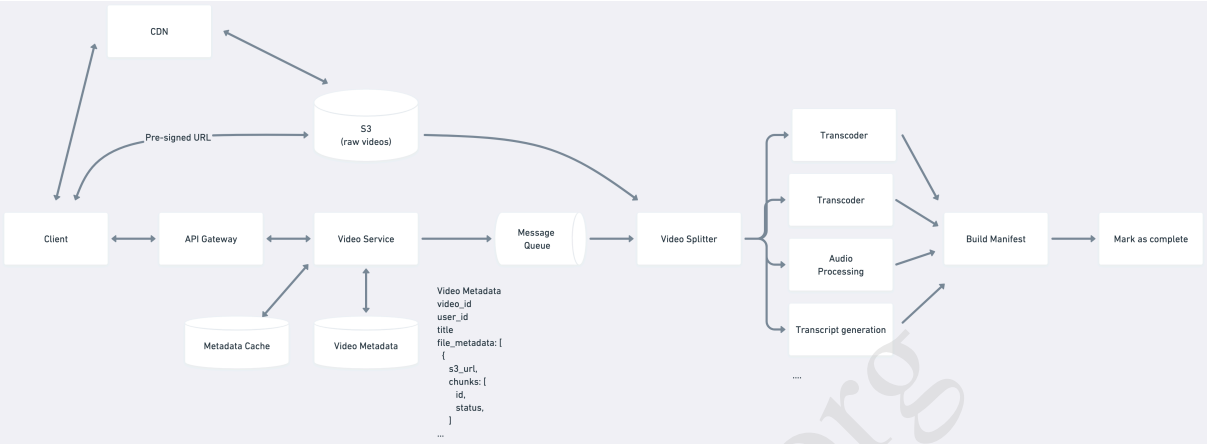
CDN 是一種分散式快取系統，會根據使用者的地理位置，從最近的節點提供內容。

當使用者發出請求時，CDN 會將請求導向最近的節點，其處理流程如下：

buildnoat.org

- 若內容已快取在該節點，CDN 直接回傳給使用者。
- 若未快取，CDN 會向 origin server（例如 S3）拉取內容、快取後再回傳。

CDN 常用於傳遞靜態內容（images、videos、HTML），也可用於動態內容（例如 API responses），能顯著降低延遲並提升串流穩定度。



buildmoat.org